04 — Entity Relationships: 1:1, 1:N, M:N

Table of Contents

Learning Objectives

After completing this section you will be able to:

- Model 1:1, 1:N, and M:N relationships correctly in SQL
- Design junction tables with attributes
- Handle self-referencing hierarchies (trees)
- Draw and interpret ER diagrams
- Choose the correct foreign key placement for each relationship type

---

Entity-Relationship Concepts

Entity: A distinguishable "thing" in the domain — users, products, orders. **Attribute:** A property of an entity — user.email, product.price. **Relationship:** A named association between entities — User PLACES Order.
Cardinality: How many instances of each entity participate:

- 1:1 — one to one
- 1:N — one to many
- M:N — many to many

Participation:

- **Mandatory (total):** Every entity must participate (|— in ER notation)
- **Optional (partial):** Entity may or may not participate (O— in ER notation)

One-to-One (1:1) Relationships

Definition

Each row in table A corresponds to at most one row in table B, and vice versa.

When it arises

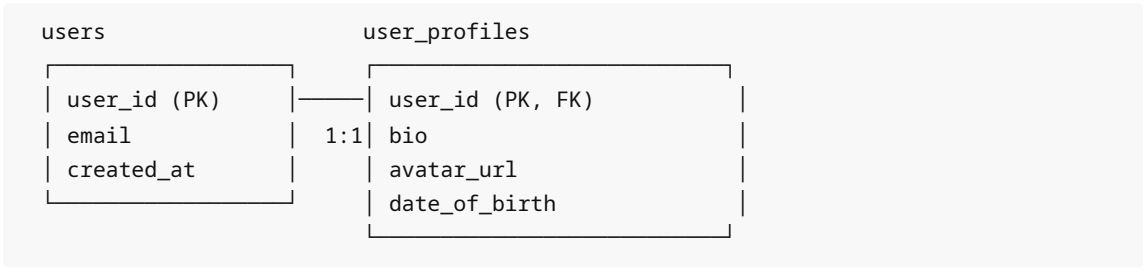
1. **Optional extension table:** Separating optional/infrequently accessed data from main table
2. **Subtype modeling:** Different subtypes of the same entity (employees that are managers)
3. **Security separation:** Storing sensitive data (passwords, SSNs) in a separate table

Implementation options

```
Option A: FK in either table (nullable = optional 1:1)
  users ──────────┐
                  │ user_profiles.user_id FK → users.user_id
  user_profiles ──┘

Option B: FK as PK (1:1 guaranteed by shared PK)
  users ──────────┐
                  │ user_secrets.user_id PK+FK → users.user_id
  user_secrets  ──┘  (PK ensures at most one secret per user)
```

ASCII ER diagram




```sql
CREATE TABLE users (
    user_id     BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email       TEXT    NOT NULL UNIQUE,
    password_hash TEXT  NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Option B: Shared PK ensures strict 1:1
CREATE TABLE user_profiles (
    user_id         BIGINT  PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
    full_name       TEXT,
    bio             TEXT,
    avatar_url      TEXT,
    date_of_birth   DATE,
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Only one profile per user; deleting user deletes profile too
INSERT INTO users (email, password_hash) VALUES ('alice@example.com', 'hash1');
INSERT INTO user_profiles (user_id, full_name) VALUES (1, 'Alice Smith');
```

One-to-Many (1:N) Relationships

Definition

One row in table A can relate to many rows in table B, but each row in B relates to only one row in A.

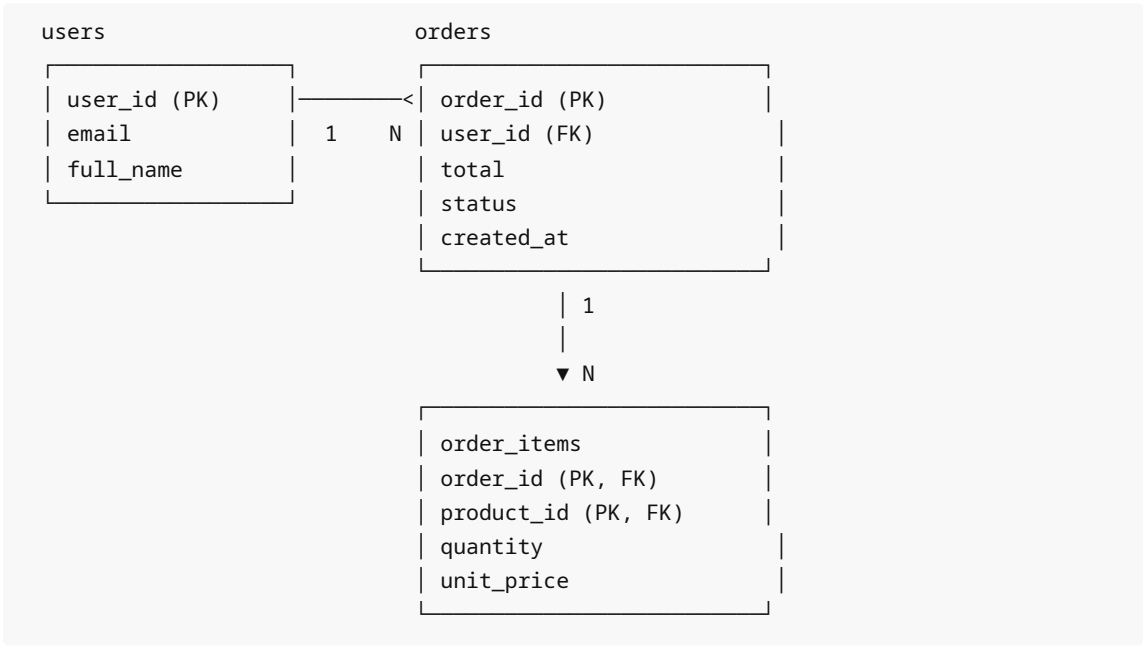
This is the **most common relationship type** in relational databases.

Implementation

Always put the FK in the "many" side (table B).

```
users (1) ──────< orders (N)
   user_id ←──── orders.user_id (FK)
```

ASCII ER diagram




```sql
CREATE TABLE departments (
    dept_id     SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        TEXT     NOT NULL UNIQUE,
    budget      NUMERIC(15,2)
);

CREATE TABLE employees (
    emp_id      BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name   TEXT    NOT NULL,
    email       TEXT    NOT NULL UNIQUE,
    dept_id     SMALLINT NOT NULL REFERENCES departments(dept_id),  -- FK on "many"
side
    salary      NUMERIC(10,2) NOT NULL CHECK (salary > 0),
    hired_at    DATE    NOT NULL DEFAULT CURRENT_DATE
);
```

```
CREATE INDEX idx_employees_dept ON employees (dept_id);  -- Essential for JOIN
performance!

-- Query: find all employees in each department
SELECT d.name AS department, count(e.emp_id) AS headcount
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.name
ORDER BY headcount DESC;
```

Many-to-Many (M:N) Relationships

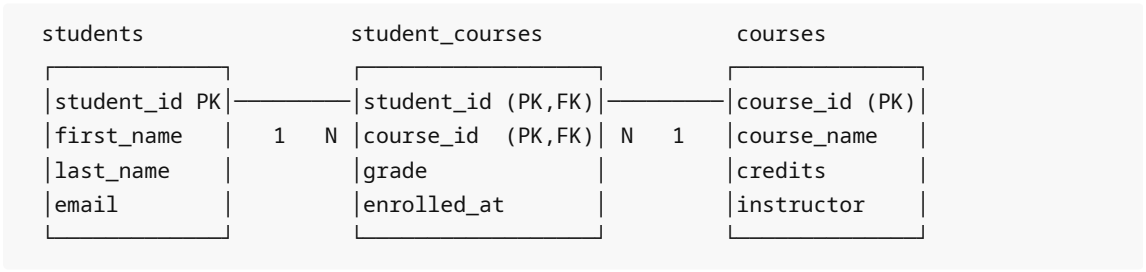
Definition

Each row in table A can relate to many rows in table B, and each row in B can relate to many rows in A.

M:N relationships cannot be directly represented in relational tables — they require a junction (associative) table.

```
students ————————< student_courses >———————— courses
  student_id           junction             course_id
```

ASCII ER diagram




```
CREATE TABLE students (
    student_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    first_name  TEXT    NOT NULL,
    last_name   TEXT    NOT NULL,
    email       TEXT    NOT NULL UNIQUE
);

CREATE TABLE courses (
    course_id   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    course_code TEXT    NOT NULL UNIQUE,
    course_name TEXT    NOT NULL,
    credits     SMALLINT NOT NULL CHECK (credits BETWEEN 1 AND 6)
);

-- Junction table (associative entity)
CREATE TABLE enrollments (
    student_id  INTEGER NOT NULL REFERENCES students(student_id) ON DELETE CASCADE,
```

```
    course_id   INTEGER NOT NULL REFERENCES courses(course_id)   ON DELETE CASCADE,
    grade       CHAR(2) CHECK (grade IN ('A+','A','A-','B+','B','B-
','C+','C','D','F')),
    enrolled_at DATE    NOT NULL DEFAULT CURRENT_DATE,
    PRIMARY KEY (student_id, course_id)
);

CREATE INDEX idx_enrollments_course ON enrollments (course_id);  -- query by course

-- Query: students in a specific course
SELECT s.first_name, s.last_name, e.grade
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
WHERE e.course_id = 42
ORDER BY s.last_name;

-- Query: all courses for a student
SELECT c.course_code, c.course_name, e.grade
FROM enrollments e
JOIN courses c ON c.course_id = e.course_id
WHERE e.student_id = 7;
```

Junction Tables (Associative Entities)

A junction table is more than a bridge — it often has its own attributes.

Rich junction table

```
-- Product tags: M:N with extra attributes
CREATE TABLE tags (
    tag_id   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name     TEXT    NOT NULL UNIQUE,
    color    TEXT    NOT NULL DEFAULT '#888888'
);

CREATE TABLE product_tags (
    product_id  INTEGER NOT NULL REFERENCES products(product_id),
    tag_id      INTEGER NOT NULL REFERENCES tags(tag_id),
    added_by    BIGINT  NOT NULL REFERENCES users(user_id),
    added_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    weight      REAL    NOT NULL DEFAULT 1.0,  -- tag relevance weight
    PRIMARY KEY (product_id, tag_id)
);

CREATE INDEX idx_product_tags_tag ON product_tags (tag_id);

-- User roles: M:N with temporal attributes
CREATE TABLE roles (
    role_id     SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        TEXT     NOT NULL UNIQUE,
```

```
    description TEXT
);

CREATE TABLE user_roles (
    user_id     BIGINT   NOT NULL REFERENCES users(user_id),
    role_id     SMALLINT NOT NULL REFERENCES roles(role_id),
    granted_by  BIGINT   REFERENCES users(user_id),
    granted_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at  TIMESTAMPTZ,  -- temporal: role can expire
    is_active   BOOLEAN  NOT NULL DEFAULT TRUE,
    PRIMARY KEY (user_id, role_id)
);

-- Active roles for a user
SELECT r.name AS role
FROM user_roles ur
JOIN roles r ON r.role_id = ur.role_id
WHERE ur.user_id = 42
  AND ur.is_active
  AND (ur.expires_at IS NULL OR ur.expires_at > now());
```

Self-Referencing Relationships

A table that has a FK to itself. Used for hierarchies (trees).

Adjacency list model (simple tree)

```
categories (self-referencing)
┌──────────────────────────────────┐
│ category_id (PK)                 │
│ name                             │◄─┐
│ parent_id (FK → category_id)     │──┘ (can be NULL for root)
└──────────────────────────────────┘

Example hierarchy:
Electronics (id=1, parent=NULL)
├── Computers (id=2, parent=1)
│   ├── Laptops (id=3, parent=2)
│   └── Desktops (id=4, parent=2)
└── Phones (id=5, parent=1)
```

```
CREATE TABLE categories (
    category_id SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        TEXT     NOT NULL,
    parent_id   SMALLINT REFERENCES categories(category_id) ON DELETE SET NULL,
    sort_order  SMALLINT NOT NULL DEFAULT 0
);

-- Recursive CTE to traverse the tree
WITH RECURSIVE category_tree AS (
```

```sql
    -- Base case: root categories (no parent)
    SELECT category_id, name, parent_id, name::TEXT AS path, 0 AS depth
    FROM categories
    WHERE parent_id IS NULL

    UNION ALL

    -- Recursive case: children
    SELECT c.category_id, c.name, c.parent_id,
           ct.path || ' > ' || c.name,
           ct.depth + 1
    FROM categories c
    JOIN category_tree ct ON ct.category_id = c.parent_id
)
SELECT category_id, path, depth
FROM category_tree
ORDER BY path;

-- Employee hierarchy (manager → employees)
CREATE TABLE employees (
    emp_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name   TEXT   NOT NULL,
    manager_id  BIGINT REFERENCES employees(emp_id) ON DELETE SET NULL
);

-- Find all reports under a manager (recursive)
WITH RECURSIVE reports AS (
    SELECT emp_id, full_name, manager_id, 0 AS level
    FROM employees
    WHERE emp_id = 5  -- top manager

    UNION ALL

    SELECT e.emp_id, e.full_name, e.manager_id, r.level + 1
    FROM employees e
    JOIN reports r ON r.emp_id = e.manager_id
)
SELECT emp_id, full_name, level
FROM reports
ORDER BY level, full_name;
```

Alternatives to adjacency list

Model	Storage	Query depth	Modify	Best for
Adjacency list	Compact	Recursive CTE	Easy	General use
Materialized path	Medium (path col)	Simple JOIN (LIKE 'x/%')	Medium	Read-heavy trees
Nested sets	Compact	Simple range	Complex	Read-only trees
ltree ext.	Medium	Path ops	Easy	Complex hierarchies

Ternary Relationships

Three entities participating in one relationship simultaneously.

```
  supplier  ──────────┐
                      │
  part      ──────────┤ supplies (ternary junction)
                      │
  project   ──────────┘
```

```
-- Ternary: supplier supplies part to project
CREATE TABLE supplier_part_project (
    supplier_id INTEGER NOT NULL REFERENCES suppliers(supplier_id),
    part_id     INTEGER NOT NULL REFERENCES parts(part_id),
    project_id  INTEGER NOT NULL REFERENCES projects(project_id),
    quantity    INTEGER NOT NULL CHECK (quantity > 0),
    unit_cost   NUMERIC(10,4) NOT NULL,
    PRIMARY KEY (supplier_id, part_id, project_id)
);
```

ER Diagram Notation

```
Crow's Foot Notation:
──┤    exactly one (mandatory one)
──O    zero or one (optional one)
──<    many
──O<   zero or many
──|<   one or many (at least one)

Examples:
  users ──|──────O< orders    (one user has zero-or-many orders)
  orders ──|──────|< order_items  (one order has one-or-many items)
  students ──O<─────O< courses  (via junction: each side zero-or-many)

Chen Notation:
  Rectangles:  entities
  Ovals:       attributes
  Diamonds:    relationships
  Double rect: weak entities
  Underline:   key attribute

Cardinality markers:
  1 ..–..N      one to many
  1 ..–.. 1     one to one
  N ..–.. M     many to many
```

SQL Examples

Complete university schema with all relationship types

```
-- 1:1: student and student_id_card
CREATE TABLE students (
    student_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name   TEXT    NOT NULL,
    email       TEXT    NOT NULL UNIQUE,
    enrolled_at DATE    NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE id_cards (
    student_id  INTEGER PRIMARY KEY REFERENCES students(student_id) ON DELETE
CASCADE,
    card_number CHAR(10) NOT NULL UNIQUE,
    issued_at   DATE     NOT NULL DEFAULT CURRENT_DATE,
    expires_at  DATE     NOT NULL DEFAULT CURRENT_DATE + INTERVAL '4 years'
);

-- 1:N: department has many courses
CREATE TABLE departments (
    dept_id  SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name     TEXT     NOT NULL UNIQUE,
    building TEXT
);

CREATE TABLE courses (
    course_id   INTEGER  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dept_id     SMALLINT NOT NULL REFERENCES departments(dept_id),
    course_code TEXT     NOT NULL UNIQUE,
    title       TEXT     NOT NULL,
    credits     SMALLINT NOT NULL DEFAULT 3
);

-- 1:N: instructor teaches many courses
CREATE TABLE instructors (
    instructor_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name     TEXT    NOT NULL,
    email         TEXT    NOT NULL UNIQUE,
    dept_id       SMALLINT REFERENCES departments(dept_id)
);

-- M:N: students enroll in many courses, each course has many students
CREATE TABLE enrollments (
    student_id    INTEGER NOT NULL REFERENCES students(student_id),
    course_id     INTEGER NOT NULL REFERENCES courses(course_id),
    semester      TEXT    NOT NULL CHECK (semester ~ '^(Spring|Summer|Fall)
\d{4}$'),
    grade         TEXT    CHECK (grade IN ('A+','A','A-','B+','B','B-
','C+','C','D','F','W')),
    enrolled_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (student_id, course_id, semester)  -- can re-take a course!
```

```
);

-- M:N: course may have prerequisite courses (self-referencing M:N)
CREATE TABLE prerequisites (
    course_id       INTEGER NOT NULL REFERENCES courses(course_id),
    prereq_course_id INTEGER NOT NULL REFERENCES courses(course_id),
    is_mandatory    BOOLEAN NOT NULL DEFAULT TRUE,
    PRIMARY KEY (course_id, prereq_course_id),
    CONSTRAINT no_self_prerequisite CHECK (course_id != prereq_course_id)
);

-- 1:N: course section (one course, many sections per semester)
CREATE TABLE sections (
    section_id    INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    course_id     INTEGER NOT NULL REFERENCES courses(course_id),
    instructor_id INTEGER NOT NULL REFERENCES instructors(instructor_id),
    semester      TEXT    NOT NULL,
    room          TEXT,
    max_capacity  SMALLINT NOT NULL DEFAULT 30
);

-- Indexes
CREATE INDEX idx_courses_dept      ON courses (dept_id);
CREATE INDEX idx_enrollments_course ON enrollments (course_id);
CREATE INDEX idx_sections_course   ON sections (course_id);
CREATE INDEX idx_sections_instructor ON sections (instructor_id);
```

Common Mistakes

1. **Putting the FK on the wrong side in 1:N**

```
-- BAD: FK on the "one" side (users has many orders)
CREATE TABLE users (order_id INTEGER REFERENCES orders(order_id));
-- This means one user can only have one order!

-- GOOD: FK on the "many" side
CREATE TABLE orders (user_id INTEGER REFERENCES users(user_id));
```

2. **Modeling M:N as separate FK arrays**

```
-- BAD: array of product IDs (no FK integrity, hard to query)
CREATE TABLE orders (product_ids INTEGER[]);

-- GOOD: junction table
CREATE TABLE order_items (order_id INTEGER, product_id INTEGER, PRIMARY
KEY(...));
```

3. **Forgetting to index the non-PK FK column in the junction table**

```
-- Junction table PK: (student_id, course_id)
-- student_id IS the leading PK column — has index
-- course_id: NOT indexed by default!
CREATE INDEX idx_enrollments_course ON enrollments (course_id);  -- REQUIRED
```

4. **Infinite loops in self-referencing tables** — Recursive CTEs can loop infinitely if there are cycles. Add `MAX_RECURSION` or use `CYCLE` clause (PostgreSQL 14+).
5. **Trying to directly create a M:N without a junction table** — Not possible in a relational model. Always use a junction table.

Best Practices

1. **Always put FK on the "many" side** in a 1:N relationship.
2. **Name junction tables** by combining the two entity names: `student_courses` not `student_course_mapping`.
3. **Index both FK columns** in junction tables.
4. **Add meaningful attributes** to junction tables when the relationship itself has properties.
5. **Use `ON DELETE CASCADE`** for dependent entities (order_items when order is deleted).
6. **Use `ON DELETE SET NULL`** for optional relationships (employee's manager is deleted).
7. **Use `ON DELETE RESTRICT`** when deletion should be prevented (deleting a category that has products).

Performance Considerations

```
-- M:N query performance depends on which direction you query most:
-- Forward (student → courses): use (student_id, course_id) as PK + index on
course_id
-- Backward (course → students): use (course_id, student_id) as PK

-- For bidirectional performance, use composite PK in one direction + extra index:
CREATE TABLE enrollments (
    student_id INTEGER NOT NULL REFERENCES students(student_id),
    course_id  INTEGER NOT NULL REFERENCES courses(course_id),
    ...
    PRIMARY KEY (student_id, course_id)   -- fast: student → courses
);
CREATE INDEX idx_enrollments_course ON enrollments (course_id, student_id);  --
fast: course → students

-- Self-referencing hierarchy performance:
-- Adjacency list: O(depth) recursive CTEs — slow for deep trees
-- Materialized path: O(1) for subtree — fast reads, slow writes
-- ltree extension: O(log n) — best general-purpose
CREATE EXTENSION IF NOT EXISTS ltree;
ALTER TABLE categories ADD COLUMN path LTREE;
CREATE INDEX idx_categories_path ON categories USING GIST (path);
```

Interview Questions & Answers

Q1: Where does the foreign key go in a 1:N relationship?

A: Always on the "many" side. If one customer has many orders, the `orders` table gets `customer_id` (FK). Never put the FK on the "one" side — that would limit one customer to one order.

Q2: How do you model a M:N relationship in a relational database?

A: With a junction (associative/bridge) table that has FKs to both parent tables. The junction table's primary key is typically the composite of both FKs. The junction table can also have its own attributes (like enrollment date, grade in a student-course enrollment).

Q3: What is a self-referencing foreign key and what is it used for?

A: A FK in a table that references the same table's PK. Used for hierarchical/tree structures: employee manager hierarchy (emp_id references emp_id as manager_id), category trees (category references parent_category), comment threads (comment references parent_comment).

Q4: How would you implement the adjacency list model for a tree, and what are its limitations?

A: Store `parent_id` as a FK to the same table. Simple to update. Query the tree with recursive CTEs (`WITH RECURSIVE`). Limitations: performance degrades with tree depth, and finding subtree sizes requires recursion. Alternative: use the `ltree` extension for path-based storage.

Q5: Should a junction table have its own surrogate PK?

A: It depends. A composite PK (both FKs) is semantically correct and prevents duplicates. A surrogate PK (extra identity column) is useful if the junction table is referenced by other tables via FK. Never add a surrogate PK just to avoid the composite PK — it adds complexity without benefit in most cases.

Q6: How would you query the "grandchildren" of a category using recursive CTE?

A: Use `WITH RECURSIVE` , start from the root category, and recurse on `parent_id = current_category_id` . Limit depth with a `depth` counter and `WHERE depth <= N` .

Q7: What is the difference between a "weak" entity and a regular entity?

A: A weak entity cannot be uniquely identified by its own attributes — it depends on a "strong" (owner) entity. Example: `order_items` (item 1 of order 1001, item 1 of order 1002 — the item number alone doesn't uniquely identify it). The PK of a weak entity includes the PK of its owner entity.

Q8: When would you use a ternary junction table instead of two binary junction tables?

A: When the three entities are simultaneously related and no combination of any two entities is sufficient to determine the relationship. The classic test: if `(supplier, part)` and `(supplier, project)` and `(part, project)` pairs are all valid independently, you might need separate binary tables. If only the three together are meaningful, use a ternary table.

Exercises with Solutions

Exercise 1

Model a library system with books, members, and loans. A member can borrow many books, a book can be borrowed by many members (over time). Include a reservation system.

Solution:

```
CREATE TABLE members (
    member_id   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name   TEXT    NOT NULL,
    email       TEXT    NOT NULL UNIQUE,
    joined_at   DATE    NOT NULL DEFAULT CURRENT_DATE,
    is_active   BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE books (
    book_id     INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    isbn        CHAR(13) NOT NULL UNIQUE,
    title       TEXT    NOT NULL,
    author      TEXT    NOT NULL,
    total_copies SMALLINT NOT NULL DEFAULT 1 CHECK (total_copies > 0)
);

-- 1:N from books: one book_id, many physical copies
CREATE TABLE book_copies (
    copy_id     INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    book_id     INTEGER NOT NULL REFERENCES books(book_id),
    condition   TEXT    NOT NULL DEFAULT 'good' CHECK (condition IN
('new','good','fair','poor'))
);

-- M:N: members borrow copies (loan records)
CREATE TABLE loans (
    loan_id     INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    copy_id     INTEGER NOT NULL REFERENCES book_copies(copy_id),
    member_id   INTEGER NOT NULL REFERENCES members(member_id),
    loaned_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    due_at      TIMESTAMPTZ NOT NULL DEFAULT now() + INTERVAL '14 days',
    returned_at TIMESTAMPTZ,
    CONSTRAINT returned_after_loaned CHECK (returned_at IS NULL OR returned_at >=
loaned_at)
);

-- M:N: reservations (members reserve books not yet available)
CREATE TABLE reservations (
    reservation_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    book_id     INTEGER NOT NULL REFERENCES books(book_id),
    member_id   INTEGER NOT NULL REFERENCES members(member_id),
    reserved_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at  TIMESTAMPTZ NOT NULL DEFAULT now() + INTERVAL '7 days',
    status      TEXT NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending','fulfilled','expired','cancelled'))
);

CREATE INDEX idx_loans_copy    ON loans (copy_id);
CREATE INDEX idx_loans_member  ON loans (member_id);
```

```
CREATE INDEX idx_loans_active  ON loans (copy_id) WHERE returned_at IS NULL;
CREATE INDEX idx_reservations_book ON reservations (book_id) WHERE status =
'pending';
```

Cross-References

- `01_normalization_1nf_2nf_3nf.md` — FK placement from normalization rules
- `05_schema_design_ecommerce.md` — real example of all relationship types
- `../05_PostgreSQL_Core/08_constraints.md` — FK cascade options
- `../07_Indexes/` — indexing FK columns